

GISA Master Data Generator

Die vollständige Projekt-Erklärung

Wie die App aufgebaut ist, wie die Dateien zusammenspielen und wie du sie selbst bauen könntest.

Für Coding-Einsteiger geschrieben. Es geht um das Verstehen der Konzepte und Zusammenhaenge – die Syntax im Detail nimmt dir der Computer ab. Nach dem Lesen sollst du erklären können, *warum* jede Datei existiert und *was* bei jedem Klick im Hintergrund passiert.

Thema	Inhalt
Technologie	SAP CAP (Node.js), OData, SAP Fiori Elements, SQLite/HANA
Aufgabe	Test-Stammdaten generieren, in SAP-Systeme pushen, tracken, kopieren, löschen
Umfang	3 Schichten (Datenbank, Service, UI) + 2 gemockte SAP-Backends
Stand	Alle Kern-Features + optionales Löschen, 17/17 Tests gruen

Lies das Dokument von vorne nach hinten – jedes Kapitel baut auf dem vorigen auf. Code-Ausschnitte sind so formatiert und bewusst kurz gehalten; du musst sie nicht auswendig können.

Inhalt

1. Das große Bild – was die App überhaupt tut
2. Grundbegriffe einfach erklärt (CAP, OData, Fiori, BTP ...)
3. Das mentale Modell: drei Schichten + externes SAP
4. Projektstruktur – welche Datei wofür da ist
5. Schicht 1: Das Datenmodell (db/schema.cds)
6. Die Daten-Pools als CSV (db/data/)
7. Schicht 2a: Der Service-Vertrag (srv/service.cds)
8. Schicht 2b: Die Service-Logik (srv/service.js)
9. Die zwei gemockten SAP-Backends (srv/external/)
10. Schicht 3a: Annotationen – das Aussehen (app/annotations.cds)
11. Schicht 3b: Die drei Fiori-Apps (app/.../webapp/)
12. Feature-Durchläufe: was bei jedem Klick passiert
13. Sicherheit & Mehrbenutzer (Auth, Rollen, \$user)
14. Tests – woher wir wissen, dass es funktioniert
15. Die wichtigsten Stolperfallen (Gotchas)
16. Bauanleitung: das Projekt selbst von null aufbauen
17. Glossar – Begriffe zum Nachschlagen

1. Das große Bild – was die App überhaupt tut

Stell dir vor, du arbeitest bei einer Firma, die SAP-Systeme betreut. Bevor neue Software live geht, muss sie getestet werden – und zwar mit **Daten, die echt aussehen**: Personen mit Namen, Adressen, Postleitzahlen, Städten. Sich diese Daten von Hand auszudenken ist mühsam und langweilig. Genau hier setzt das Projekt an.

Der **GISA Master Data Generator** ist eine Web-Anwendung, die solche Test-Stammdaten **automatisch erzeugt** und sie dann per Knopfdruck in ein oder mehrere SAP-Systeme überträgt. Konkret kann die App:

- **Generieren**: Aus Daten-Pools (Listen von Vornamen, Nachnamen, Straßen, Städten ...) werden zufällige, realistische Datensätze zusammengewürfelt.
- **Pushen**: Die erzeugten Daten werden über eine standardisierte Schnittstelle (OData) in ein SAP-Backend geschrieben – als Business Partner mit Adresse.
- **Tracken**: Die App merkt sich, *welches* Objekt in *welchem* System mit *welchem* Schlüssel angelegt wurde (ein Protokoll).
- **Mehrere Systeme**: Es gibt nicht nur ein Ziel, sondern mehrere SAP-Systeme, die man verwalten und gezielt ansteuern kann.
- **Kopieren**: Daten, die in System A liegen, können nach System B übertragen werden.
- **Löschen**: Was angelegt wurde, kann auch wieder aus dem SAP-System entfernt werden.

Das Ganze hat eine grafische Oberfläche (UI), damit man nicht programmieren muss, um die Funktionen zu nutzen – man klickt Buttons.

Merke

Die App ist im Kern eine **Brücke**: Auf der einen Seite erzeugt sie Daten aus Pools, auf der anderen Seite spricht sie SAP-Systeme an. Alles andere (Tracking, Kopieren, Löschen, mehrere Systeme) sind Komfort- und Verwaltungsfunktionen drumherum.

2. Grundbegriffe einfach erklärt

Bevor wir in die Dateien schauen, ein paar Begriffe – bewusst in Alltagssprache.

SAP

Ein riesiges Software-Unternehmen. Viele große Firmen verwalten ihre Geschäftsdaten (Kunden, Rechnungen, Material ...) in SAP-Systemen. Unser „Business Partner“ ist so ein Geschäftsdaten-Objekt: eine Person oder Firma, mit der man Geschäfte macht.

SAP BTP (Business Technology Platform)

Die **Cloud-Plattform** von SAP, auf der man eigene Apps betreiben kann – vergleichbar mit einem Hosting-Anbieter, aber speziell für die SAP-Welt. Unsere App ist dafür gebaut, auf der BTP zu laufen.

SAP CAP (Cloud Application Programming Model)

Das **Baukasten-Framework**, mit dem unsere App geschrieben ist. CAP gibt dir eine feste Struktur vor (wo das Datenmodell hingehört, wo die Logik, wo die UI) und nimmt dir sehr viel Standard-Arbeit ab. Statt eine Datenbank-Verbindung, eine REST-API und die Web-Schnittstelle alles selbst zu programmieren, beschreibst du nur, *was* du willst – CAP baut das *wie* größtenteils automatisch. CAP-Projekte laufen auf **Node.js** (JavaScript auf dem Server).

Wichtiges CAP-Prinzip: „Convention over Configuration“;

CAP erkennt Dinge an **festen Ordnernamen**. Was in db/ liegt, ist das Datenmodell. Was in srv/ liegt, sind die Services (die Logik). Was in app/ liegt, ist die Oberfläche. Du musst CAP also nicht erklären, wo was ist – du haeltst dich nur an die Konvention, und es funktioniert.

CDS (Core Data Services)

Die kleine, eigene **Beschreibungssprache** von CAP (Dateien mit Endung .cds). Damit beschreibt man Tabellen („Entities“) und Services in wenigen Zeilen. CDS ist *deklarativ*: du sagst, *was* existieren soll, nicht *wie* es technisch umgesetzt wird.

OData

Ein **standardisierter Weg, über das Internet mit Daten zu reden** – eine besondere Art von Web-Schnittstelle (API). Wenn unsere App Daten in SAP anlegt, schickt sie OData-Anfragen (z.B. „erzeuge einen neuen Business Partner mit diesen Feldern“). Der große Vorteil: SAP-Systeme „sprechen“ OData, und CAP erzeugt für unseren eigenen Service automatisch auch eine OData-Schnittstelle. Beide Seiten reden also dieselbe Sprache.

SAP Fiori / Fiori Elements

Fiori ist der Design-Stil von SAP-Oberflächen (das einheitliche Aussehen aller modernen SAP-Apps). **Fiori Elements** ist ein cleverer Trick: Statt jede Tabelle und jedes Formular von Hand zu programmieren, beschreibt man nur per **Annotationen** (Markierungen am Datenmodell), welche Felder wie angezeigt werden sollen – und Fiori Elements baut die fertige Oberfläche daraus. Darunter läuft die Technik **SAPUI5** (SAPs JavaScript-Baukasten für Oberflächen).

SQLite und HANA

Zwei **Datenbanken**. **SQLite** ist winzig und liegt einfach als Datei auf dem Laptop – perfekt zum lokalen Entwickeln. **HANA** ist die große Datenbank von SAP für den echten Betrieb in der Cloud. Das Schöne an CAP: Dieselbe App läuft lokal auf SQLite und in der Cloud auf HANA, **ohne dass man den Code ändern muss**.

Git, Branch, Commit

Git ist die „Versionsverwaltung“ – ein Werkzeug, das jede Änderung am Code speichert, sodass man nichts verliert und zu jedem Stand zurück kann. Ein **Commit** ist ein gespeicherter Schnappschuss. Ein **Branch** ist eine Abzweigung, auf der man ein neues Feature in Ruhe baut, ohne den Hauptstand kaputtzumachen; ist es fertig, wird der Branch in den Hauptstand „gemergt“ (zusammengeführt).

3. Das mentale Modell: drei Schichten + externes SAP

Fast jede Daten-App ist in **Schichten** aufgebaut. Wenn du dieses Bild im Kopf hast, ordnet sich jede Datei automatisch ein. Unsere App hat drei eigene Schichten und spricht ausserdem mit externen SAP-Systemen:



Die **goldene Regel**: Anfragen fließen von oben nach unten (UI ruft Service, Service liest/schreibt Daten), Antworten fließen von unten nach oben zurück. Die UI redet **nie direkt** mit der Datenbank – immer über den Service. Das hält die Logik an einer Stelle und macht die App sicher und wartbar.

Warum „externe“ Backends?

Das eigentliche Ziel der App ist, Daten in **fremde SAP-Systeme** zu schreiben. Diese Systeme gehören nicht zu unserer App. Beim lokalen Entwickeln haben wir aber kein echtes SAP zur Hand – deshalb **simulieren** („mocken“) wir zwei davon. Für unsere App fühlt sich das genauso an wie echte Systeme; nur läuft die Simulation mit auf dem Laptop.

4. Projektstruktur – welche Datei wofür da ist

Hier der Ordnerbaum mit dem Zweck jeder wichtigen Datei. Diese Tabelle ist dein „Inhaltsverzeichnis des Codes“ – bei Unklarheit immer hierher zurück.

```

Generator/
|
+-- db/                                SCHICHT 1: Datenmodell + Startdaten
|   +-- schema.cds                    Definition aller eigenen Tabellen
|   +-- data/                         Start-Inhalte als CSV (Pools, Systeme)
|       +-- gisa.mdg-Cities.csv, ...-FirstNames.csv, ...-Systems.csv
|
+-- srv/                               SCHICHT 2: Services (Logik)
|   +-- service.cds                   "Vertrag": welche Daten/Aktionen gibt es
|   +-- service.js                    "Code": was passiert bei jeder Aktion
|   +-- external/                     Die simulierten SAP-Backends
|       +-- _mockBackend.js           gemeinsame Simulations-Logik
|       +-- BackendAPI_2.csn/.edmx/.js Backend 1 (System "S4D")
|       +-- BackendAPI_3.csn/.js      Backend 2 (System "S4Q")
|
+-- app/                               SCHICHT 3: Oberflaeche (UI)
|   +-- annotations.cds               Wie sehen die Tabellen/Spalten aus
|   +-- generator/webapp/             App 1: Daten generieren + alle Aktionen
|   +-- tracking/webapp/              App 2: Protokoll der angelegten Objekte
|   +-- systems/webapp/               App 3: Zielsysteme verwalten
|
+-- test/integration.test.js          Automatische Tests (17 Stueck)
+-- package.json                     Projekt-Steckbrief + Konfiguration
+-- HANDOFF.md                       Kurz-Uebergabe fuer neue Mitarbeiter

```

Jede der drei Fiori-Apps hat denselben inneren Aufbau (dazu später mehr):

- `manifest.json` – der Bauplan der App (welche Tabelle, welche Buttons).
- `Component.js` – der Startpunkt der App (winzig, immer fast gleich).
- `index.html` – die HTML-Seite, die alles startet.
- `ext/...js` – eigener JavaScript-Code für die Buttons.
- `i18n/i18n.properties` – Texte (Titel etc.), zentral an einer Stelle.

5. Schicht 1: Das Datenmodell (db/schema.cds)

Hier wird beschrieben, welche **Tabellen** es in unserer eigenen Datenbank gibt. In CDS heißt eine Tabelle **Entity**. Schauen wir uns die wichtigsten an.

5.1 Die Daten-Pools

Pools sind einfache Listen, aus denen später zufällig gezogen wird. Beispiel:

```
db/schema.cds (Auszug)
namespace gisa.mdg; // gemeinsamer "Nachname" aller Tabellen

using { cuid } from '@sap/cds/common'; // bringt eine fertige ID-Spalte mit

entity FirstNames : cuid { // Tabelle "FirstNames", erbt cuid
    firstName : String(50) @mandatory;
}
entity StreetNames : cuid { streetName : String(100) @mandatory; }
entity Cities : cuid { cityName : String(100) @mandatory; }
// ... LastNames, Neighborhoods, PostCodes, HouseNumbers analog
```

Was hier passiert, Zeile für Zeile:

- `namespace gisa.mdg;` – ein Präfix, damit alle Tabellen eindeutig „gisa.mdg.FirstNames“ usw. heißen. Verhindert Namens-Kollisionen.
- `using { cuid }` – wir leihen uns einen fertigen Baustein von CAP. `cuid` ergaenzt jede Tabelle automatisch um eine Spalte ID, die eine weltweit eindeutige Zufallsnummer (UUID) bekommt.
- `: cuid` hinter dem Entity-Namen heißt „erbe die ID-Spalte“ (Vererbung, wie ein Bausatz).
- `String(50)` – Text mit höchstens 50 Zeichen. `@mandatory` – Pflichtfeld, darf nicht leer sein.

Aha-Moment

Du *beschreibst* nur die Tabelle – du schreibst nirgends „CREATE TABLE“. CAP liest diese Beschreibung und legt die echte Datenbank-Tabelle selbst an. Das ist der Kern von „deklarativ“: sagen was, nicht wie.

5.2 Die Ergebnis-Tabelle: GeneratorData

Hier landen die fertig zusammengewuerfelten Datensätze, bevor sie ins SAP gepusht werden.

```
db/schema.cds (Auszug)
entity GeneratorData {
    key concatID : String; // eindeutiger Schluessel der Zeile
    streetName : String(100);
    cityName : String(100);
    firstName : String(50);
    lastName : String(50);
    postCode : String(5);
    houseNumber : String(3);
    // ... neighborhoodName
    createdBy : String(255); // WER hat diese Zeile erzeugt (Login-Name)
}
```

- `key concatID` – das Wort `key` macht dieses Feld zum **Primärschlüssel** (die eindeutige Kennung jeder Zeile). `concatID` wird aus den IDs der gezogenen Pool-Einträge zusammengesetzt („concatenated“ = verkettet).
- `createdBy` – der Login-Name des Nutzers. Damit weiss die App, **wem** eine Zeile gehört – wichtig, wenn mehrere Leute gleichzeitig arbeiten (dazu Kapitel 13).

5.3 Die Tracking-Tabelle: CreatedObjects

Das Protokoll: welches Objekt wurde in welchem System mit welchem Schlüssel angelegt. Genau diese Tabelle hat die Aufgabenstellung verlangt (Spalten System | Objekt | Key).

`db/schema.cds` (Auszug)

```
entity CreatedObjects : cuid {
  system      : String(100); // Ziel-System, z.B. "S4D"
  objectType  : String(50);  // "Street"/"City"/"Address"/"BusinessPartner"
  objectKey   : String(100); // die vom SAP vergebene Nummer
  sourceConcatID : String;    // Verweis auf die GeneratorData-Zeile
  createdBy   : String(255); // wer hat gepusht
  createdAt   : Timestamp;   // wann
}
```

Diese Tabelle ist das Bindeglied für fast alles Spätere: **Kopieren** und **Löschen** schauen hier nach, was ein Nutzer in einem System angelegt hat.

5.4 Die Systeme-Tabelle: Systems

Der Katalog der SAP-Zielsysteme. Ohne diese Tabelle gäbe es nur ein einziges festes Ziel.

`db/schema.cds` (Auszug)

```
entity Systems : cuid {
  name       : String(20) @mandatory; // Kuerzel, z.B. "S4D"
  description : String(200);
  serviceName : String(100) @mandatory; // welcher CAP-Service spricht das System an
  isDefault   : Boolean default false;  // Standard-Ziel beim Push
}
```

Der entscheidende Trick steckt in `serviceName`: Dieser Text sagt, **über welchen konfigurierten Verbindungskanal** das System angesprochen wird („BackendAPI_2“ oder „BackendAPI_3“). So kann der Code zur Laufzeit das richtige Backend wählen, nur anhand eines Tabellen-Eintrags.

6. Die Daten-Pools als CSV (db/data/)

Eine leere Tabelle nützt nichts – die Pools brauchen Inhalt. Den liefern **CSV-Dateien** (einfache Textdateien mit Komma-getrennten Werten, wie eine Mini-Tabelle). CAP lädt sie automatisch in die passende Tabelle, **wenn der Dateiname zum Tabellennamen passt**.

Beispiel-CSV

Dateiname: `db/data/gisa.mdg-Cities.csv`

```
ID,cityName
1cty,Albrechts bei Suhl
2cty,Allersdorf bei Ilmenau
...
```


Die erste Zeile sind die **Spaltennamen** (müssen exakt den Feldern im Schema entsprechen), darunter die Datenzeilen. Der Dateiname gisa.mdg-Cities.csv verrät CAP: „Das gehört in die Tabelle gisa.mdg.Cities“.

Auch die **Zielsysteme** werden so vorbefüllt – dadurch existieren S4D und S4Q sofort, ohne dass jemand sie manuell anlegen muss:

Seed der Zielsysteme

Dateiname: db/data/gisa.mdg-Systems.csv

ID,name,description,serviceName,isDefault

s4d,S4D,SAP S/4HANA Development (Mock),BackendAPI_2,true

s4q,S4Q,SAP S/4HANA Quality (Mock),BackendAPI_3,false

Begriff: „Seed“

Daten, mit denen eine frische Datenbank von Anfang an befüllt wird, nennt man **Seed-Daten** („Saat“). Pools und die zwei Systeme sind unser Seed.

7. Schicht 2a: Der Service-Vertrag (srv/service.cds)

Der Service ist das Tor zur Aussenwelt. Die `.cds`-Datei ist sein **Vertrag**: Sie legt fest, *welche* Daten nach aussen sichtbar sind und *welche Aktionen* es gibt – aber noch nicht, was die Aktionen tun (das kommt in der `.js`-Datei).

```
srv/service.cds (gekuerzt)
using { gisa.mdg as my } from '../db/schema.cds'; // Datenmodell einbinden

@path : '/service/generator' // unter dieser URL ist der Service erreichbar
@requires : 'Generator' // nur Nutzer mit Rolle "Generator" duerfen rein
service GeneratorService {

    entity StreetNames as projection on my.StreetNames; // Pool sichtbar machen
    // ... weitere Pools

    // Jeder sieht/aendert nur SEINE eigenen Zeilen:
    @restrict: [{ grant: '*', to: 'Generator', where: 'createdBy = $user' }]
    entity GeneratorData as projection on my.GeneratorData;

    @readonly // Tracking nur lesen, nicht aendern
    @restrict: [{ grant: 'READ', to: 'Generator', where: 'createdBy = $user' }]
    entity CreatedObjects as projection on my.CreatedObjects;

    entity Systems as projection on my.Systems; // gemeinsam gepflegt

    // AKTIONEN (Knöpfe, die etwas tun):
    action generateTestCustomers(anzahl : Integer) returns String;
    action pushToBackend(system : String) returns String;
    action copyData(sourceSystem : String, targetSystem : String) returns String;
    action deleteFromBackend(system : String) returns String;
}
```

Die wichtigen Begriffe darin

- **projection on** – eine „Durchreiche“: Die interne Tabelle `my.GeneratorData` wird nach aussen als Service-Tabelle sichtbar gemacht. Man kann dabei filtern oder Felder verstecken – hier reichen wir sie weitgehend 1:1 durch.
- **@path** – die Internet-Adresse des Service. Alles läuft unter `/service/generator`.
- **@requires : 'Generator'** – ein Türsteher: Nur eingeloggte Nutzer mit der Rolle „Generator“ dürfen den Service überhaupt benutzen.
- **@restrict ... where: 'createdBy = \$user'** – eine Zeilen-Filter-Regel. `$user` ist der aktuell eingeloggte Nutzer. Die Regel sagt: „Zeige jedem nur die Zeilen, deren `createdBy` sein eigener Name ist.“ So sehen sich mehrere Nutzer nicht gegenseitig in die Daten.
- **@readonly** – diese Tabelle darf von aussen nur gelesen, nicht verändert werden (das Tracking-Protokoll füllt nur der Server selbst).
- **action ... returns String** – eine aufrufbare Funktion mit Eingaben in Klammern und einer Rückgabe (hier eine Text-Meldung). Das sind die vier Knöpfe der App.

Vertrag vs. Umsetzung

Die `.cds` sagt nur **dass** es z.B. `pushToBackend` gibt und welche Eingaben es nimmt. **Wie** der Push tatsächlich abläuft, steht in der `.js`-Datei (nächstes Kapitel). Diese Trennung – Vertrag hier, Umsetzung dort – ist ein wiederkehrendes Muster in CAP.

8. Schicht 2b: Die Service-Logik (srv/service.js)

Diese Datei ist das **Gehirn** der App. Hier steht echter JavaScript-Code, der bei jeder Aktion ausgeführt wird. Der Grundaufbau:

```
srv/service.js (Geruest)

const cds = require('@sap/cds');           // CAP-Werkzeugkasten laden

module.exports = class GeneratorService extends cds.ApplicationService {
  async init() {

    // Kurznamen fuer die Tabellen holen:
    const { GeneratorData, CreatedObjects, Systems, /* Pools... */ } = this.entities;

    // Fuer jede Aktion ein "Handler" = was bei Aufruf passiert:
    this.on('generateTestCustomers', async (req) => { /* ... */ });
    this.on('pushToBackend',         async (req) => { /* ... */ });
    this.on('copyData',              async (req) => { /* ... */ });
    this.on('deleteFromBackend',     async (req) => { /* ... */ });

    return super.init();
  }
}
```

- **this.on('name', handler)** – „Wenn die Aktion *name* aufgerufen wird, fuehre diese Funktion aus.“ Das Muster heit **Event-Handler** (auf ein Ereignis reagieren).
- **async / await** – Datenbank- und Netzwerk-Aufrufe dauern einen Moment. `await` heit „warte hier, bis das Ergebnis da ist“, ohne die ganze App zu blockieren. `async` markiert eine Funktion, die solche Wartepunkte enthalten darf.
- **req** (request) – das Anfrage-Objekt. Darin steckt u.a. `req.data` (die Eingaben, z.B. `anzahl`) und `req.user.id` (wer fragt).

8.1 Aktion „Generieren“

Ziel: `anzahl` zufällige Datensätze bauen und in `GeneratorData` speichern. Vereinfachter Ablauf:

```

srv/service.js (sinngemaess)
this.on('generateTestCustomers', async (req) => {
  const anzahl = req.data.anzahl || 10;
  const owner = req.userId; // wer generiert

  // 1. Alte eigene Zeilen weg (nur die eigenen!):
  await DELETE.from(GeneratorData).where({ createdBy: owner });

  // 2. Alle Pools aus der DB lesen:
  const streets = await SELECT.from(StreetNames);
  const cities = await SELECT.from(Cities);
  // ... firstNames, lastNames, postCodes, houseNumbers, neighborhoods

  // 3. anzahl-mal zufaellig kombinieren:
  const entries = [];
  for (let i = 0; i < anzahl; i++) {
    const s = streets[Math.floor(Math.random() * streets.length)];
    const c = cities[Math.floor(Math.random() * cities.length)];
    // ... f, l, p, h analog
    entries.push({ concatID: s.ID+'-'+c.ID+'-...', streetName: s.streetName,
                    cityName: c.cityName, /* ... */ createdBy: owner });
  }

  // 4. Alle auf einmal speichern:
  await INSERT.into(GeneratorData).entries(entries);
  return `Successfully generated ${anzahl} customers.`;
});

```

- **SELECT / INSERT / DELETE** – CAPs eingebaute Befehle, um mit der Datenbank zu reden. Sie lesen fast wie Englisch: `SELECT.from(X)`, `INSERT.into(X).entries(...)`, `DELETE.from(X).where(...)`.
- **Math.random()** liefert eine Zufallszahl zwischen 0 und 1; damit ziehen wir einen zufaelligen Eintrag aus jedem Pool.
- Wir löschen nur `where createdBy = owner` – also nur die eigenen alten Zeilen, nie die von anderen.
- Wir sammeln erst alles in `entries` und speichern **einmal** – das ist schneller, als jede Zeile einzeln zu schreiben.

8.2 Aktion „Pushen“ (mit Zielsystem-Wahl)

Ziel: die eigenen GeneratorData-Zeilen ins gewählte SAP-Backend schreiben – und jedes angelegte Objekt im Tracking festhalten.

srv/service.js (gekuerzt)

```
this.on('pushToBackend', async (req) => {  
  // 1. Zielsystem bestimmen: gewaehltes ODER das Default-System:  
  const sys = req.data.system  
    ? await SELECT.one.from(Systems).where({ ID: req.data.system })  
    : await SELECT.one.from(Systems).where({ isDefault: true });  
  
  // 2. Verbindung zum passenden Backend aufbauen:  
  const backend = await cds.connect.to(sys.serviceName); // "BackendAPI_2" o. "_3"  
  
  // 3. Eigene Zeilen holen:  
  const owner = req.user.id;  
  const rows = await SELECT.from(GeneratorData).where({ createdBy: owner });  
  
  const tracked = [];  
  for (const cust of rows) {  
    // 4. Im Backend anlegen - Reihenfolge wegen Verknuepfungen:  
    const street = await backend.create('Street').entries({ name: cust.streetName });  
    const city = await backend.create('City').entries({ name: cust.cityName });  
    const addr = await backend.create('Address').entries({  
      street_ID: street.ID, city_ID: city.ID,  
      houseNumber: toBackendHouseNumber(cust.houseNumber), postalCode: cust.postCode });  
    const bp = await backend.create('BusinessPartner').entries({  
      firstName: cust.firstName, surName: cust.lastName, address_ID: addr.ID });  
  
    // 5. Jedes Objekt fuer Tracking vormerken (Key = vom Backend vergebene Nummer):  
    tracked.push({ system: sys.name, objectType: 'BusinessPartner',  
      objectKey: String(bp.businessPartnerNumber), createdBy: owner });  
    // ... ebenso fuer Street, City, Address  
  }  
  // 6. Tracking-Zeilen speichern:  
  await INSERT.into(CreatedObjects).entries(tracked);  
  return `Successfully pushed ${rows.length} customers to ${sys.name}.`;  
});
```

Die Schluessestellen verstanden

- **cds.connect.to(sys.serviceName)** – baut eine Verbindung zum externen Backend auf. Welches, steht im Tabellen-Eintrag des Systems. **Genau hier** wird aus „mehrere Systeme“ Realitaet: je nach Wahl verbindet sich der Code zu einem anderen Ziel.
- **Reihenfolge Street -> City -> Address -> BusinessPartner:** Eine Adresse verweist auf Straße und Stadt; ein Business Partner verweist auf eine Adresse. Man muss also zuerst das anlegen, worauf später verwiesen wird (street.ID wird in der Adresse gebraucht).
- **objectKey = businessPartnerNumber:** Die *Nummern* (Schlüssel) vergibt das SAP-Backend selbst beim Anlegen. Wir lesen sie aus der Antwort und schreiben sie ins Tracking – so wissen wir später genau, was wir wo erzeugt haben.
- **toBackendHouseNumber(...):** Das Backend verlangt Hausnummern im Format „Ziffern + Kleinbuchstabe“ (z.B. 12a). Eine kleine Hilfsfunktion bringt unsere reinen Zahlen in dieses Muster.

8.3 Aktion „Kopieren“

Ziel: die eigenen Business Partner aus System A lesen und in System B neu anlegen. Der Clou: Wir wissen aus dem **Tracking**, welche BPs uns in System A gehören.

- 1 Quell- und Zielsystem aus Systems nachschlagen (und prüfen, dass sie verschieden sind).
- 2 Im Tracking die eigenen BusinessPartner-Schlüssel im Quellsystem suchen.

- 3 Mit diesen Schlüsseln die vollständigen Datensätze (BP, Adresse, Straße, Stadt) aus dem Quell-Backend lesen – **flach**, Schritt für Schritt (der simulierte Server kann keine tief verschachtelten Abfragen).
- 4 Im Ziel-Backend alles neu anlegen (gleiche Reihenfolge wie beim Push).
- 5 Die neuen Objekte unter dem Zielsystem-Namen tracken.

Warum über das Tracking statt „alles kopieren“?

Wenn man stumpf *alle* BPs aus System A kopieren würde, kopierte man auch die Daten anderer Nutzer. Indem wir nur die im Tracking als *eigene* markierten Schlüssel nehmen, bleibt das Kopieren **mehrbenutzer-sicher**. Das Tracking ist also nicht nur Protokoll, sondern aktives Werkzeug.

8.4 Aktion „Löschen“

Ziel: die eigenen, im System angelegten Objekte wieder aus dem Backend entfernen und das Tracking aufräumen.

- 1 Im Tracking alle eigenen Einträge für das gewählte System holen.
- 2 Pro Objekttyp die gespeicherten Nummern nehmen und damit im Backend die zugehörigen Datensätze (ihre internen IDs) finden.
- 3 Diese über ihren Schlüssel löschen – in der Reihenfolge BusinessPartner -> Address -> Street -> City (zuerst das, was auf anderes verweist).
- 4 Zuletzt die Tracking-Einträge dieses Nutzers/Systems entfernen.

Auch hier ist das Tracking die Grundlage: Es liefert die Schlüssel, die zum gezielten Löschen nötig sind.

Das wiederkehrende Muster

Push, Copy und Delete folgen alle demselben Dreischritt: **(1)** herausfinden, was zu tun ist (aus DB/Tracking lesen), **(2)** mit dem Backend reden (anlegen/lesen/löschen), **(3)** das Tracking aktualisieren. Wer ein Feature versteht, versteht alle.

9. Die zwei gemockten SAP-Backends (srv/external/)

Diese Dateien **simulieren** echte SAP-Systeme, damit man lokal entwickeln und testen kann, ohne ein echtes SAP. „Mock“ bedeutet Attrappe.

9.1 Woraus ein Backend besteht

- **BackendAPI_2.edmx / .csn** – die **Beschreibung** der Backend-Tabellen (Business Partner, Address, Street, City) und ihrer Felder. Das .edmx ist das Original-Format von SAP; das .csn ist die von CAP genutzte Fassung.
- **BackendAPI_2.js** – die **Simulations-Logik** dieses Backends.
- **BackendAPI_3.csn / .js** – ein **zweites** Backend, fast identisch, aber mit eindeutigen Namen. Dadurch sind es wirklich zwei getrennte Datentöpfe (Voraussetzung dafür, dass „Kopieren von A nach B“ überhaupt sichtbar wird).

9.2 Gemeinsame Logik: _mockBackend.js

Beide Backends verhalten sich gleich, also steht die Logik **einmal** in `_mockBackend.js` und wird von beiden genutzt (kein Doppel-Code). Sie ahmt zwei Verhaltensweisen des echten SAP nach:

- **Nummern vergeben:** Echtes SAP vergibt beim Anlegen automatisch eindeutige Nummern (z.B. businessPartnerNumber). Der Mock würfelt eine solche Nummer, damit sich alles echt anfühlt.
- **Validieren:** Das echte SAP lehnt ungültige Eingaben ab. Der Mock prüft per Muster, dass die Hausnummer wie 12a und die PLZ aus genau 5 Ziffern besteht – sonst gibt es einen Fehler (Code 400).

srv/external/_mockBackend.js (Auszug)

```
// _mockBackend.js (Kerngedanke)
const nextNumber = () => Math.floor(Math.random() * 9_999_999_999) + 1;
const HOUSE = /^[0-9]{1,4}[a-z]$/; // erlaubtes Hausnummer-Muster
const PLZ = /^[0-9]{5}$/; // erlaubtes PLZ-Muster

this.before('CREATE', Street, req => { req.data.streetNumber ??= nextNumber(); });
this.before('CREATE', Address, req => {
  req.data.addressNumber ??= nextNumber();
  if (!HOUSE.test(req.data.houseNumber)) req.error(400, 'Hausnummer ungültig');
  if (!PLZ.test(req.data.postalCode)) req.error(400, 'PLZ ungültig');
});
```

- **this.before('CREATE', ...)** – „Tu das, *bevor* ein neuer Datensatz angelegt wird.“ Hier: Nummer setzen bzw. Eingaben prüfen.
- **/^[0-9]{5}\$/** ist ein **regulärer Ausdruck** („Regex“) – ein Such-Muster für Text. Hier: „genau fünf Ziffern“.
- **??=** heißt „setze nur, falls noch leer“.

9.3 Wie die Backends bekannt gemacht werden

Damit der Code `cds.connect.to('BackendAPI_2')` sagen kann, müssen die Backends in `package.json` als Verbindungen eingetragen sein:

package.json (Auszug)

```
// package.json -> cds -> requires
"BackendAPI_2": { "kind": "odata", "model": "srv/external/BackendAPI_2" },
"BackendAPI_3": { "kind": "odata", "model": "srv/external/BackendAPI_3" }
```

kind: 'odata' sagt: „Sprich mit diesem System per OData.“ Lokal wird es gemockt, in der Cloud würde hier die Adresse des echten SAP stehen – **der App-Code bliebe gleich.**

10. Schicht 3a: Annotationen – das Aussehen (app/annotations.cds)

Erinnerung an Fiori Elements: Man programmiert die Oberfläche nicht, man **beschreibt** sie. Diese Beschreibungen heißen **Annotationen** und stehen in app/annotations.cds. Beispiel für die Tracking-Tabelle:

app/annotations.cds (Auszug)

```
// Welche Spalten in welcher Reihenfolge in der Tabelle erscheinen:
annotate GeneratorService.CreatedObjects with @UI.LineItem: [
  { $Type: 'UI.DataField', Value: system },
  { $Type: 'UI.DataField', Value: objectType },
  { $Type: 'UI.DataField', Value: objectKey },
  { $Type: 'UI.DataField', Value: createdAt }
];

// Schoenere Spalten-Ueberschriften:
annotate GeneratorService.CreatedObjects with {
  system      @title: 'System';
  objectType  @title: 'Objekttyp';
  objectKey   @title: 'Schluessel';
};
```

- **@UI.LineItem** – die Liste der Spalten einer Tabelle (engl. „line item“ = Zeilen-Eintrag). Die Reihenfolge hier ist die Reihenfolge in der Oberfläche.
- **@title** – die Beschriftung, die der Nutzer sieht (statt des technischen Feldnamens).
- **@UI.SelectionFields** (nicht gezeigt) – welche Felder als Filter oben erscheinen.

Der Aha-Moment bei Fiori Elements

Es gibt **keinen** handgeschriebenen Code, der die Tabelle, die Filter oder die Detailseite baut. Fiori Elements liest diese Annotationen und erzeugt daraus die komplette, voll funktionsfähige Oberfläche. Du beschreibst nur das **Was** – das **Wie** macht das Framework.

11. Schicht 3b: Die drei Fiori-Apps (app/.../webapp/)

Es gibt drei kleine Oberflächen, die sich die Daten teilen (sie reden alle mit demselben Service):

App	Ordner	Zweck
Generator	app/generator/webapp/	Daten generieren; alle Aktions-Buttons (Push, Copy, Delete)
Tracking	app/tracking/webapp/	Protokoll der angelegten Objekte ansehen
Systeme	app/systems/webapp/	Zielsysteme ansehen / neu anlegen / löschen

11.1 manifest.json – der Bauplan

Sagt der App, welche Datenquelle sie nutzt, welche Tabelle sie zeigt und welche **eigenen Buttons** es gibt:

```

app/generator/webapp/manifest.json (Auszug)
"datasources": { "mainService": { "uri": "/service/generator/" } },
...
"entitySet": "GeneratorData",          // diese Tabelle zeigt die App
...
"controlConfiguration": {
  "@...UI.LineItem": { "actions": {
    "push": { "press": "...GeneratorActions.onPush", "text": "An Backend pushen" },
    "copy": { "press": "...GeneratorActions.onCopy", "text": "Daten kopieren" },
    "deleteInSystem": { "press": "...onDelete", "text": "Im System loeschen" }
  }}
}

```

Jeder Button verweist per press auf eine Funktion in unserer eigenen JavaScript-Datei. Klickt der Nutzer, wird genau diese Funktion aufgerufen.

11.2 ext/GeneratorActions.js – was die Buttons tun

Hier steht der Code hinter den Buttons. Beispiel „Pushen“: erst die Systeme laden, dann einen Dialog mit Auswahl zeigen, dann die Service-Aktion aufrufen.

```

app/generator/webapp/ext/GeneratorActions.js (gekuerzt)
onPush: function () {
  loadSystems().then(function (systeme) {
    // ... Dialog mit Dropdown der Systeme bauen ...
    // beim Klick auf "Pushen":
    callAction("pushToBackend", { system: gewaehlteId })
      .then(msg => MessageToast.show(msg)); // Erfolgsmeldung anzeigen
  });
},

// Hilfsfunktion: ruft eine Service-Aktion per OData auf
function callAction(name, body) {
  return fetch("/service/generator/" + name, {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body)
  }).then(r => r.json()).then(t => t.value);
}

```

- **fetch(...)** – der Standard-Weg im Browser, eine Anfrage ans Internet/an den Server zu schicken. Hier ein POST an die Aktion – das ist der OData-Aufruf.
- **MessageToast.show(msg)** – blendet kurz die Rückgabe-Meldung der Aktion ein („Successfully pushed 3 customers to S4D.“).
- **Dialog mit Dropdown** – bauen wir hier selbst in JavaScript, weil wir dem Nutzer eine Auswahl (Zielsystem, Von/Nach) geben wollen.

Die Navigation zwischen den drei Apps passiert einfach über Links auf die jeweils andere Adresse, z.B. `window.location.href = '/tracking/webapp/index.html'`.

11.3 index.html – der Start (und eine wichtige Falle)

Diese HTML-Seite startet die App. Sie enthält absichtlich etwas mehr Code als üblich, um einen typischen Anfänger-Fehler zu vermeiden:

Gotcha: die „weiße Seite“;

Wenn die App in einen Bereich mit **Höhe 0** geladen wird, sieht man nur eine weiße Seite – die App ist da, aber unsichtbar zusammengequetscht. Lösung (im Projekt drin): per CSS bekommen Seite und Container `height : 100%`, und die App wird explizit mit voller Höhe gestartet. Diese Falle kostet sonst Stunden Fehlersuche.

12. Feature-Durchläufe: was bei jedem Klick passiert

Jetzt verbinden wir alle Schichten. Hier der komplette Weg eines Klicks – von der Oberfläche bis zur Datenbank und zurück.

12.1 „Generieren“

```
[Nutzer] Klick "Generieren" -&gt; Eingabe Anzahl (z.B. 5)
|
v
[UI]   GeneratorActions.onGenerate -&gt; fetch POST .../generateTestCustomers
|
v
[Service]  service.js Handler 'generateTestCustomers'
|          - löscht eigene alte Zeilen
|          - liest alle Pools
|          - wuerfelt 5x zusammen
|          - INSERT in GeneratorData
v
[Datenbank] 5 neue Zeilen (createdBy = ich)
|
v
[UI]   Tabelle aktualisiert sich -&gt; 5 Zeilen sichtbar + Erfolgsmeldung
```

12.2 „Pushen“ (Multi-System)

```
[Nutzer] Klick "An Backend pushen" -&gt; Dialog: Zielsystem S4Q -&gt; "Pushen"
|
v
[UI]   onPush -&gt; fetch POST .../pushToBackend { system: "s4q" }
|
v
[Service] Handler 'pushToBackend'
|          - Systems nachschlagen: s4q -&gt; serviceName "BackendAPI_3"
|          - cds.connect.to("BackendAPI_3")
|          - fuer jede eigene Zeile:
|              Street -&gt; City -&gt; Address -&gt; BusinessPartner anlegen
|              Nummern aus Antwort merken
|          - INSERT der Tracking-Zeilen (system = "S4Q")
v
[Backend S4Q] legt Objekte an, vergibt Nummern
[Datenbank]   CreatedObjects: neue Zeilen mit system="S4Q"
|
v
[UI]   Meldung "Successfully pushed N customers to S4Q."
```

12.3 „Kopieren“ und „Löschen“

Beide folgen demselben Muster, nur mit anderer Backend-Operation:

```
KOPIEREN (Von S4D Nach S4Q):
Tracking lesen: meine BPs in S4D -&gt; Daten aus S4D lesen
-&gt; in S4Q neu anlegen -&gt; Tracking fuer S4Q schreiben

LOESCHEN (in S4D):
Tracking lesen: meine Objekte in S4D -&gt; im S4D-Backend per Key loeschen
-&gt; Tracking-Eintraege fuer S4D entfernen
```

12.4 Tracking ansehen

Die Tracking-App ist eine reine Lese-Ansicht: Sie fragt über OData die Tabelle CreatedObjects ab (gefiltert auf den eigenen Nutzer durch die @restrict-Regel) und zeigt sie als Fiori-Tabelle – ganz ohne zusätzlichen Code, nur dank Annotationen.

Der rote Faden

CreatedObjects (das Tracking) ist das Herz, das alle Features verbindet: Push füllt es, die Tracking-App zeigt es, Copy und Delete lesen daraus. Verstehst du diese eine Tabelle, verstehst du das Zusammenspiel der ganzen App.

13. Sicherheit & Mehrbenutzer (Auth, Rollen, \$user)

Die App ist für mehrere Nutzer gleichzeitig gedacht. Drei Bausteine sorgen dafür, dass das sicher und sauber funktioniert:

- **Authentifizierung („Auth“):** Wer bist du? Man muss sich einloggen. Lokal beim Entwickeln nutzt CAP einen einfachen Test-Login („dummy“), in der Cloud das echte SAP-Login-System (XSUAA).
- **Rolle 'Generator':** Nur wer diese Rolle hat, darf den Service überhaupt benutzen (@requires). Ohne Login: Fehler 401. Eingeloggt, aber ohne Rolle: Fehler 403.
- **\$user-Filter:** Durch `where: 'createdBy = $user'` sieht und verändert jeder nur seine eigenen Zeilen. Wenn Alice generiert, löscht sie nicht aus Versehen die Daten von Bob.

Genau dieses `createdBy`-Feld zieht sich durch: beim Generieren gesetzt, beim Anzeigen gefiltert, beim Pushen/Kopieren/Löschen als „gehört mir“-Markierung genutzt.

14. Tests – woher wir wissen, dass es funktioniert

Es gibt 17 **automatische Tests** (Datei `test/integration.test.js`). Ein Test startet die App, führt echte Aktionen aus und prüft das Ergebnis – alles per Knopf (`npm test`). Beispiele, was geprüft wird:

- Ohne Login wird eine Aktion abgelehnt (401); mit Login ohne Rolle ebenfalls (403).
- Generieren erzeugt *genau* die angeforderte Anzahl Zeilen.
- Zwei Nutzer sehen nur ihre eigenen Daten und löschen sich nicht gegenseitig.
- Push nach S4Q landet wirklich im zweiten Backend und wird als „S4Q“ getrackt.
- Kopieren von S4D nach S4Q legt die Datensätze im Ziel an.
- Löschen entfernt die Objekte und räumt das Tracking; im leeren System gibt es Fehler 400.

Warum Tests wichtig sind

Tests sind ein **Sicherheitsnetz**. Änderst du später etwas und ein Test wird rot, weißt du sofort, dass du etwas kaputtgemacht hast – bevor es ein Nutzer merkt. „17/17 gruen“ heißt: alle 17 Prüfungen bestanden.

15. Die wichtigsten Stolperfallen (Gotchas)

Diese Punkte haben beim Bauen echt Zeit gekostet – gut, sie zu kennen:

Falle	Was passiert / Lösung
Weiße Seite (Höhe 0)	Fiori-App rendert unsichtbar. Lösung: <code>index.html</code> startet die App explizit mit voller Höhe + <code>CSS height:100%</code> .
Buttons tun nichts	Fiori ruft eigene Aktionen nicht automatisch auf. Lösung: Buttons im manifest definieren + eigene Handler (<code>ext/*.js</code>) die per <code>fetch</code> aufrufen.
Lokaler Push schlägt fehl	'no such table' bzw. 'no credentials': Server muss mit <code>'--with-mocks --in-memory'</code> laufen, sonst fehlen die Mock-Tabellen.
Kopieren bricht ab	Der Mock kann keine tief verschachtelten Abfragen (<code>\$expand</code>). Lösung: flach in Schritten lesen (BP, dann Adresse, dann Straße/Stadt).

Falle	Was passiert / Lösung
Harmlose Konsolen-'Fehler'	Einige 404/Warnungen im Browser sind im Entwicklungsmodus normal und können ignoriert werden.

16. Bauanleitung: das Projekt selbst von null aufbauen

Wenn du es selbst nachbauen wolltest, wäre das die sinnvolle Reihenfolge. Jeder Schritt baut auf dem vorigen auf – genau so ist das Projekt auch entstanden.

- 1 Projekt anlegen:** Mit dem CAP-Werkzeug ein leeres Projekt erzeugen (`cds init`). Es gibt dir die Ordner `db/` `srv/` `app/` vor.
- 2 Datenmodell schreiben** (`db/schema.cds`): die Pool-Tabellen, `GeneratorData`. Erst später `CreatedObjects` und `Systems` ergaenzen.
- 3 Pools mit CSV befüllen** (`db/data/`): pro Tabelle eine CSV mit passendem Namen.
- 4 Service-Vertrag schreiben** (`srv/service.cds`): Tabellen sichtbar machen, Rolle/Filter setzen, die Aktionen deklarieren.
- 5 Service-Logik schreiben** (`srv/service.js`): mit `generateTestCustomers` starten – das läuft komplett ohne Backend und gibt schnell ein Erfolgserlebnis.
- 6 Lokal starten** (`cds watch`) und im Browser ausprobieren. CAP baut die Datenbank automatisch.
- 7 Backend anbinden:** ein externes OData-Backend (echt oder gemockt) in `package.json` eintragen, dann `pushToBackend` bauen.
- 8 Oberfläche verschönern:** Annotationen (`app/annotations.cds`) und eine Fiori-App anlegen; eigene Buttons über `manifest` + `Handler`.
- 9 Erweitern:** Tracking-Tabelle, dann mehrere Systeme, dann Kopieren, dann Löschen – jeweils Schema + Service + UI + Test.
- 10 Absichern:** für jedes Feature einen automatischen Test schreiben.

Die wichtigste Gewohnheit

Schritt für Schritt, klein halten, nach jedem Schritt testen. Lieber zehnmal etwas Kleines, das funktioniert, als einmal etwas Grosses, das nirgends läuft. Genau so wurde dieses Projekt gebaut – ein Feature pro Branch, getestet, dann zusammengeführt.

17. Glossar – Begriffe zum Nachschlagen

Begriff	Kurz erklärt
CAP	SAPs Baukasten-Framework für Daten-Apps (läuft auf Node.js).
CDS	Beschreibungssprache von CAP für Tabellen und Services (.cds-Dateien).
Entity	Eine Tabelle im Datenmodell.
Service	Das Tor nach aussen: stellt Daten und Aktionen bereit.
Action	Eine aufrufbare Funktion eines Service (ein 'Knopf').
Handler	Code, der bei einem Ereignis (z.B. Aktion) ausgeführt wird.

Begriff	Kurz erklärt
OData	Standard-Web-Schnittstelle, über die mit Daten geredet wird.
Projection	Eine Durchreiche/Sicht einer internen Tabelle nach aussen.
Annotation	Markierung am Modell, die das UI-Aussehen beschreibt (@UI...).
Fiori Elements	Erzeugt fertige Oberflächen aus Annotationen.
Mock	Eine Attrappe/Simulation eines echten Systems zum Testen.
Seed	Startdaten, mit denen eine frische DB befüllt wird (CSV).
cuid	Fertiger Baustein, der jeder Tabelle eine eindeutige ID gibt.
UUID	Eine weltweit eindeutige Zufalls-Kennung.
\$user	Platzhalter für den aktuell eingeloggtten Nutzer.
Rolle	Berechtigung eines Nutzers (hier: 'Generator').
async/await	Sprachmittel, um auf langsame Aufrufe zu warten.
Branch / Commit	Git: Abzweigung zum Entwickeln / gespeicherter Stand.

Geschafft. Wenn dir die drei Schichten, der Weg eines Klicks und die Rolle der Tracking-Tabelle klar sind, hast du das Projekt im Kern verstanden – und könntest es Schritt für Schritt selbst nachbauen.